

Unit 2

Supervised Learning

1. Definition:

Supervised learning is a type of machine learning where the model learns from **labeled training data**. Each training example consists of:

- **Input features (X)** – The attributes describing an instance.
- **Output labels (Y)** – The correct classification or value associated with each instance.

The goal of supervised learning is to learn a **mapping function $f: X \rightarrow Y$** that can accurately predict Y for new inputs.

2. Types of Supervised Learning:

1. **Classification** – The output Y is a **discrete category** (e.g., spam or not spam).
 - Example: **Email spam detection** (Spam/Not Spam).
 - Algorithms: **Naïve Bayes, Decision Trees, SVM, Neural Networks**.
2. **Regression** – The output Y is a **continuous value** (e.g., predicting house prices).
 - Example: **Stock price prediction**.
 - Algorithms: **Linear Regression, Polynomial Regression, Neural Networks**.

3. Steps in Supervised Learning:

1. **Collect Labeled Data** – Obtain a dataset with input-output pairs.
2. **Preprocess the Data** – Handle missing values, normalize features, and encode categorical variables.
3. **Split the Data** – Divide into **training** and **testing** sets (e.g., 80%-20%).
4. **Train the Model** – Use training data to adjust model parameters.
5. **Evaluate Performance** – Use metrics like **accuracy, precision, recall, RMSE** on test data.
6. **Make Predictions** – Use the trained model on new unseen data.

4. Example: Supervised Learning for Spam Classification

Email	Contains "Free"?	Contains "Win"?	Spam (1) / Not Spam (0)
1	Yes	No	1
2	No	Yes	1
3	No	No	0

- **Model:** Naïve Bayes Classifier.
- **Training:** The model learns **word patterns** that indicate spam.
- **Prediction:** For a new email, the model predicts **Spam (1) or Not Spam (0)**.

5. Common Supervised Learning Algorithms:

- **Logistic Regression** – Used for binary classification.
- **Decision Trees & Random Forests** – Handle both classification and regression.
- **Support Vector Machines (SVM)** – Effective for high-dimensional data.
- **Neural Networks** – Powerful models for image and speech recognition.

6. Advantages of Supervised Learning:

- **Highly accurate** if trained on good-quality data.
- **Interpretable models** like decision trees and linear regression.
- **Generalizes well** when sufficient data is available.

7. Limitations of Supervised Learning:

- **Requires labeled data**, which can be expensive to obtain.
- **Struggles with unseen patterns** if training data is insufficient.
- **Computationally expensive** for large datasets.

Classification & Regression

Classification and regression are the two main types of **supervised learning** tasks. The key difference is in the nature of the output:

- **Classification** → Predicts **discrete labels/categories** (e.g., spam or not spam).
- **Regression** → Predicts **continuous numerical values** (e.g., house price prediction).

1. Classification

Definition: Classification assigns input data to **predefined categories or classes**.

Example: Email Spam Classification

Email	Contains "Win"?	Contains "Offer"?	Spam (1) / Not Spam (0)
1	Yes	No	1
2	No	Yes	1
3	No	No	0

- **Goal:** Predict whether an email is **Spam (1) or Not Spam (0)**.
- **Algorithm:** Naïve Bayes, Decision Tree, Support Vector Machine (SVM).

Types of Classification:

1. **Binary Classification:** Two possible labels (e.g., **spam or not spam**).
2. **Multi-Class Classification:** More than two classes (e.g., **dog, cat, bird**).

Evaluation Metrics for Classification:

- **Accuracy** – Overall correctness of predictions.
- **Precision & Recall** – Useful when dealing with imbalanced data.
- **F1-Score** – Balances precision and recall.

2. Regression

Definition: Regression predicts **continuous numerical values** based on input features.

Example: House Price Prediction

House	Size (sq ft)	Bedrooms	Price (\$)
1	1200	3	250,000
2	1500	4	320,000
3	1800	4	400,000

- **Goal:** Predict house price based on **size and number of bedrooms**.
- **Algorithm:** Linear Regression, Polynomial Regression, Decision Trees.

Types of Regression:

1. **Linear Regression** – Fits a straight-line relationship.
2. **Polynomial Regression** – Fits a curved relationship.
3. **Logistic Regression** – Used for **binary classification** (despite its name).

Evaluation Metrics for Regression:

- **Mean Squared Error (MSE)** – Measures average squared differences.
- **Root Mean Squared Error (RMSE)** – Square root of MSE for interpretability.
- **R-squared (R^2)** – Explains variance captured by the model.

3. Key Differences Between Classification & Regression

Feature	Classification	Regression
Output Type	Discrete labels (categories)	Continuous values (numerical)
Example	Spam detection, disease diagnosis	House price prediction, stock forecasting
Algorithms	Decision Trees, SVM, Naïve Bayes	Linear Regression, Random Forest Regression
Metrics	Accuracy, Precision, Recall	MSE, RMSE, R^2

Performance Metrics for Classification Problems

Evaluating a classification model requires various **performance metrics** to measure how well it predicts class labels. Different metrics help assess accuracy, precision, recall, and the trade-off between them.

1. Confusion Matrix

A **confusion matrix** is a table that summarizes the performance of a classification model by comparing predicted vs. actual labels.

Actual / Predicted	Positive (1)	Negative (0)
Positive (1)	True Positive (TP)	False Negative (FN)
Negative (0)	False Positive (FP)	True Negative (TN)

- **True Positive (TP):** Correctly classified positive instances.
- **True Negative (TN):** Correctly classified negative instances.
- **False Positive (FP):** Incorrectly classified negatives as positives (**Type I Error**).
- **False Negative (FN):** Incorrectly classified positives as negatives (**Type II Error**).

2. Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Best for balanced datasets** but **misleading for imbalanced classes** (e.g., 95% accuracy in a dataset where 95% belong to one class).

3. Precision (Positive Predictive Value, PPV)

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Useful when False Positives are costly** (e.g., spam detection, medical diagnosis).
- High precision means **fewer false alarms**.

4. Recall (Sensitivity, True Positive Rate, TPR)

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Useful when False Negatives are costly** (e.g., fraud detection, cancer diagnosis).
- High recall means **fewer missed cases**.

5. F1-Score (Harmonic Mean of Precision and Recall)

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **Balances Precision and Recall** (useful when both errors matter).

6. Specificity (True Negative Rate, TNR)

$$\text{Specificity} = \frac{TN}{TN + FP}$$

- Measures **how well negatives are identified** (useful in medical tests).

7. ROC Curve & AUC (Area Under Curve)

- **Receiver Operating Characteristic (ROC) Curve** plots **TPR vs. FPR**.
- **AUC (Area Under the Curve)** measures overall performance.
- **Higher AUC means better classification ability.**

8. Logarithmic Loss (Log Loss)

$$\text{Log Loss} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

- Used for **probabilistic classifiers** (e.g., Logistic Regression, Neural Networks).

9. Matthews Correlation Coefficient (MCC)

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

- **Best for imbalanced datasets** (unlike accuracy).

Performance Metrics for Regression Problems

Regression models predict **continuous values**, so evaluation metrics measure the difference between predicted and actual values. These metrics assess **error, variance, and goodness-of-fit**.

1. Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Measures the **average absolute error** between actual (y_i) and predicted ($\hat{y}_i$) values.
- **Easy to interpret** but does not penalize large errors heavily.

2. Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Penalizes **larger errors more** than MAE (due to squaring).
- Sensitive to **outliers**.

3. Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- Similar to MSE but **in the same units as yyy**.
- **Popular metric** for real-world applications.

4. R-Squared (R^2 , Coefficient of Determination)

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

- Measures how well the model **explains variance** in the data.
- $R^2 = 1 \rightarrow$ Perfect fit, $R^2 = 0 \rightarrow$ No predictive power.
- Can be **misleading** for non-linear models.

5. Adjusted R-Squared (R_{adj}^2)

$$R_{adj}^2 = 1 - \left(\frac{(1 - R^2)(n - 1)}{n - k - 1} \right)$$

- **Penalizes unnecessary features** (better for multiple regression).

6. Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

- Expresses error as **percentage**.
- **Problematic for small y_i values** (division by small numbers inflates error).

7. Mean Squared Logarithmic Error (MSLE)

$$\text{MSLE} = \frac{1}{n} \sum_{i=1}^n (\log(1 + y_i) - \log(1 + \hat{y}_i))^2$$

- **Useful when target values vary exponentially** (e.g., income, population).
- Penalizes **under-predictions more than over-predictions**.

8. Huber Loss

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2, & \text{for } |a| \leq \delta \\ \delta(|a| - \frac{1}{2}\delta), & \text{for } |a| > \delta \end{cases}$$

- **Combines MAE & MSE**: Uses **MSE for small errors** and **MAE for large errors** (robust to outliers).

Linear Regression

1. Definition:

Linear Regression is a supervised learning algorithm used for **predicting continuous values** by modeling the relationship between input features and the target variable using a straight-line equation.

It assumes a **linear relationship** between the input X and the output Y :

$$Y = w_0 + w_1X_1 + w_2X_2 + \dots + w_nX_n + \epsilon$$

where:

- Y = Predicted output (dependent variable).
- $X_1, X_2, \dots, X_n$, = Input features (independent variables).
- w_0 = Intercept (bias term).
- $w_1, w_2, \dots, w_n$, = Weights (coefficients for each feature).
- ϵ = Error term (captures unmodeled noise).

2. Types of Linear Regression:

1. **Simple Linear Regression** – One independent variable:

$$Y = w_0 + w_1X + \epsilon$$

Example: Predicting house price based on size.

2. **Multiple Linear Regression** – Multiple independent variables:

$$Y = w_0 + w_1X_1 + w_2X_2 + \dots + w_nX_n + \epsilon$$

Example: Predicting house price based on **size, number of rooms, and location**.

3. Finding the Best Fit Line

Linear regression **optimizes** the weights w to minimize the error between predicted and actual values. This is done using:

- **Ordinary Least Squares (OLS) Method:** Minimizes **Mean Squared Error (MSE)**:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i is the actual value and $\hat{y}_i$ is the predicted value.

- **Gradient Descent Algorithm:** Iteratively updates weights w to minimize error using:

$$w_j := w_j - \alpha \frac{\partial}{\partial w_j} MSE$$

where α is the **learning rate**.

4. Assumptions of Linear Regression:

- **Linearity:** The relationship between X and Y is linear.

- **Independence:** Data points are independent.
- **Homoscedasticity:** Constant variance of residuals (errors).
- **No Multicollinearity:** Features should not be highly correlated.

5. Evaluation Metrics:

- **R-Squared (R^2)** → Measures variance explained by the model.
- **Mean Absolute Error (MAE)** → Measures average absolute error.
- **Mean Squared Error (MSE)** → Penalizes larger errors.
- **Root Mean Squared Error (RMSE)** → Square root of MSE for interpretability.

6. Applications of Linear Regression:

- **House price prediction** (based on size, location, etc.).
- **Stock market forecasting** (using historical prices)
- **Medical cost estimation** (based on patient data).
- **Sales forecasting** (based on advertising spend).

7. Limitations of Linear Regression:

- **Assumes linearity** – Cannot capture complex relationships.
- **Sensitive to outliers** – Can significantly affect predictions.
- **Requires feature selection** – Irrelevant variables reduce accuracy.

Decision Trees

1. Definition:

A **decision tree** is a supervised learning model used for **classification** and **regression**. It represents decisions as a **tree-like structure**, where:

- **Nodes** represent features (attributes).
- **Edges** represent decision rules.
- **Leaves** represent class labels (classification) or numerical values (regression).

2. Structure of a Decision Tree

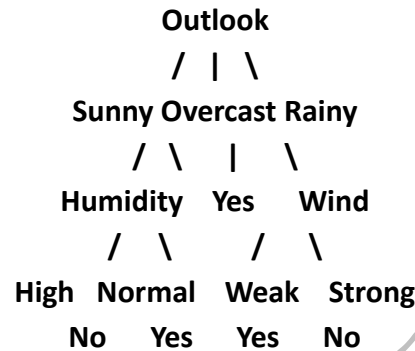
1. **Root Node** → Represents the best feature to split on first.
2. **Internal Nodes** → Represent subsequent feature splits.
3. **Leaf Nodes** → Represent final predictions (class label or value).

3. Example: Classifying Whether a Person Plays Tennis

Outlook	Temperature	Humidity	Wind	Play Tennis
Sunny	Hot	High	Weak	No

Outlook	Temperature	Humidity	Wind	Play Tennis
Sunny	Cool	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Rainy	Cool	Normal	Weak	Yes

Decision Tree:



- If **Outlook = Overcast**, **Play Tennis = Yes**.
- If **Outlook = Sunny** and **Humidity = High**, **Play Tennis = No**.

4. Decision Tree Learning Algorithm (ID3, C4.5, CART)

1. **Select the Best Feature to Split** (e.g., **Outlook**).
2. **Split the Dataset** based on the feature values.
3. **Recursively Repeat** for each subset until all data is classified.
4. **Stopping Condition**:
 - If all samples belong to one class → **Stop Splitting**.
 - If no features left → **Use Majority Class**.

5. Splitting Criteria: Choosing the Best Feature

- **Information Gain (IG) – Used in ID3 Algorithm**

$$IG(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

- Measures how well a feature separates data.
- Higher **information gain** → Better feature split.
- **Gini Impurity – Used in CART Algorithm**

$$Gini(D) = 1 - \sum_{i=1}^c p_i^2$$

- Measures classification impurity.
- Lower Gini score → Better feature split.

- **Chi-Square and Gain Ratio** – Used in C4.5.

6. Decision Trees for Regression (CART Algorithm)

- Instead of classification, decision trees predict **continuous values** by minimizing **Mean Squared Error (MSE)** at each split.

7. Advantages of Decision Trees

- **Simple and easy to interpret.**
- **Handles categorical and numerical data.**
- **Requires little preprocessing (no need for feature scaling).**
- **Captures non-linear relationships.**

8. Limitations of Decision Trees

- **Prone to overfitting** (deep trees memorize training data).
- **Sensitive to noisy data** (small changes affect splits).
- **Greedy splitting** → May not find the optimal tree.

9. How to Improve Decision Trees?

- **Pruning** – Remove unnecessary branches to prevent overfitting.
- **Ensemble Methods** – Use **Random Forest** (multiple trees) for better accuracy.
- **Regularization** – Limit tree depth to improve generalization.

10. Applications of Decision Trees

- **Medical Diagnosis** – Classifying diseases based on symptoms.
- **Fraud Detection** – Identifying fraudulent transactions.
- **Customer Segmentation** – Predicting customer behavior.
- **Credit Scoring** – Approving or rejecting loan applications.

Overfitting

1. Definition:

Overfitting occurs when a machine learning model learns the **noise and random fluctuations** in the training data instead of the underlying pattern. As a result, the model performs **very well on training data but poorly on unseen test data**.

2. Causes of Overfitting

- **High Model Complexity** – Too many parameters (e.g., deep decision trees, complex neural networks).
- **Insufficient Training Data** – Model memorizes the small dataset instead of generalizing.
- **Noisy Data** – Model captures random variations instead of meaningful patterns.

- **Too Many Features** – Irrelevant features increase complexity and reduce generalization.

3. Example of Overfitting in Decision Trees

- **Deep Decision Tree** → Perfectly classifies training data, but **fails on new data**.
- **Shallow Pruned Tree** → Generalizes better by ignoring minor details.

4. How to Detect Overfitting?

- **High Training Accuracy, Low Test Accuracy** – If test performance drops significantly, overfitting is likely.
- **Cross-Validation Performance** – If k-fold cross-validation gives inconsistent results, overfitting may be present.
- **Learning Curve Analysis** – A large gap between training and validation loss indicates overfitting.

5. Techniques to Prevent Overfitting

1. Regularization

- **L1 Regularization (Lasso)** → Shrinks less important features to zero.
- **L2 Regularization (Ridge)** → Penalizes large coefficients to simplify the model.

2. Pruning (For Decision Trees)

- **Pre-pruning** – Stop growing the tree if splits do not significantly reduce impurity.
- **Post-pruning** – Remove branches that add little value after tree construction.

3. Cross-Validation

- Use **k-fold cross-validation** to ensure generalization.

4. Early Stopping (For Neural Networks)

- Stop training when **validation loss starts increasing**, preventing unnecessary memorization.

5. Reduce Model Complexity

- Use **simpler models** (e.g., shallow decision trees, linear models).
- **Feature Selection** – Remove irrelevant features to avoid overfitting.

6. Increase Training Data

- More data helps the model generalize better.
- **Data Augmentation** – Artificially generate new data (e.g., rotating images in computer vision).

7. Ensemble Methods

- **Bagging (Bootstrap Aggregating)** – Uses multiple models to reduce variance (e.g., Random Forest).
- **Boosting** – Sequentially improves weak models (e.g., AdaBoost, Gradient Boosting).